

연락처 관리 웹 서비스

TRD (기술 요구사항 정의서) · 2 차 과제

항목	내용
문서명	연락처 관리 웹 서비스(2 차 과제) TRD (기술 요구사항 정의서)
문서 유형	Technical Requirements Document
과제 구분	2 차 과제 — FastAPI + DB 연락처 프로그램
선행 문서	00 과제목적 · 01 구현요구사항 · 02 화면정의서 · 03 기능정의서 · 04 PRD
버전	v1.2 (실행 검증판)
언어/환경	Python 3.12+ / FastAPI 0.139.0 / SQLAlchemy 2.0.51 / PostgreSQL 16 (Docker)
상태	확정(Baseline)

이 문서의 역할(안내) TRD 는"어떻게 만드는가(How, 기술적으로)"를 정의합니다. 04 PRD 가"사용자가 무엇을 할 수 있어야 하는가"를 다뤘다면, 본 문서는 그것을 구현하는 아키텍처, 데이터베이스 설계(DDL·FK), 인증 설계(세션), 모듈 설계, 기술 스택 버전, 배포/운영 방식을 정의합니다. 01(구현요구사항)·03(기능정의서)에서 이미 다른 내용은 기술적 근거와 구체적 스펙(DDL, 시퀀스, 설정값)으로 심화합니다.

개정 이력 (v1.0 → v1.1)

구분	v1.0 문제	v1.1 수정
인증(§4-3, §5-4)	LoginSession 에 user relationship 이 없어 session.user 호출 시 AttributeError 발생	User/LoginSession 모델에 relationship("User") 명시 추가
DB 설계(§4)	DDL 과 SQLAlchemy 모델을 둘 다 실행 주체처럼 제시해 스키마 miss-match 위험	SQLAlchemy 모델을 단일 진실 원천으로 명시, DDL 은 참고용으로 표기
에러 메시지(§5-4)	세션 만료 개념이 없는데 "만료" 문구 사용	"로그인이 필요합니다"로 통일
설정 관리(§3, §7-2)	python-dotenv 없이 .env 사용 전제 → KeyError 위험	기술 스택에 python-dotenv 추가, load_dotenv() 호출 명시
DB 설계(§4-2)	UNIQUE 복합 인덱스와 중복되는 단독 인덱스 존재	중복 인덱스 제거, 이름 검색용 인덱스만 유지

개정 이력 (v1.1 → v1.2) — 실제 구현·실행 검증 결과 반영

v1.1 문서를 실제 코드(FastAPI+SQLAlchemy+SQLite 로 대체 실행) 및 브라우저(Playwright)로 회원가입·로그인 화면까지 직접 실행해 검증했다.

구분	v1.1 문제	v1.2 수정
DB 설계(\$4-3)	Contact 모델에 category relationship 이 없어, \$6-1 의 category_name 응답 필드를 만들 때 AttributeError 발생 (연락처 추가/조회 API 전체가 500 으로 죽음) — session.user 와 동일 패턴의 누락	Contact 모델에 relationship("Category") 명시 추가
DB 설계(\$4-2, 참고)	회원 탈퇴 등으로 users 가 CASCADE 삭제되면, 연락처가 남아있는 categories 도 RESTRICT 없이 함께 삭제됨을 실측으로 확인 (동일 트랜잭션에서 contacts 가 먼저 CASCADE 되므로)	현재 범위(회원 탈퇴 미구현)에서는 문제 없음을 확인. \$9 향후 확장에 회원 탈퇴 기능을 추가할 경우 별도 확인 절차가 필요하다는 점만 참고로 기록
화면(\$4 화면정의서 SCR-001)	로그인 화면의 회원가입→로그인 전환 흐름을 실제 브라우저로 미검증 상태였음	Playwright 로 회원가입(201)→로그인(200)→관리 화면 전환→새로고침 유지→로그아웃까지 전 과정 실행 확인, 문제 없음

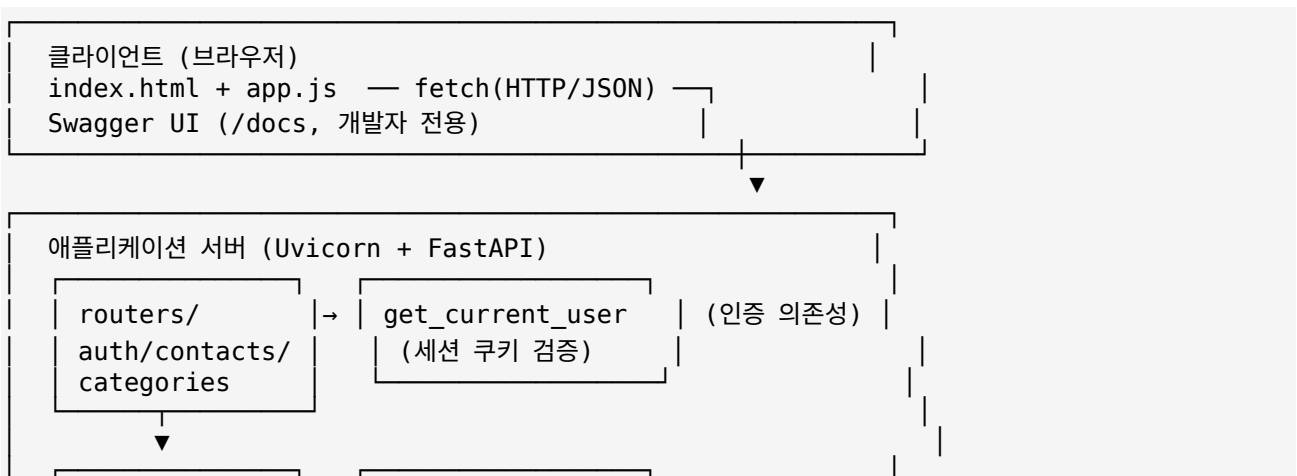
1. 개요 및 설계 원칙

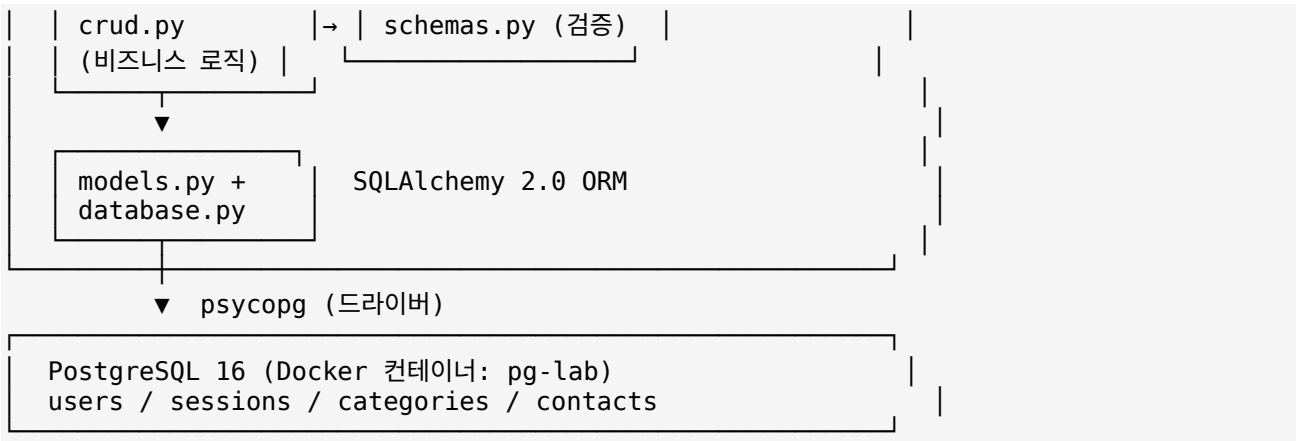
본 시스템은 1 차 과제(콘솔 + pickle)의 기능을 유지하면서, 다음 5 가지 기술적 목표를 달성하는 것을 설계 원칙으로 삼는다.

- 계층 분리: 화면(HTML/JS) — API(FastAPI 라우터) — 비즈니스 로직(crud) — 데이터(PostgreSQL) 4 계층을 명확히 분리한다.
- 무상태(Stateless) API + 세션 기반 인증: API 자체는 요청마다 독립적으로 처리되며, "누가 요청했는가"만 세션 쿠키로 식별한다.
- 데이터 무결성 우선: 애플리케이션 코드보다 DB 제약(PK/FK/UNIQUE)을 신뢰의 최종 방어선으로 둔다.
- 예측 가능한 오류 응답: 모든 예외는 상태 코드 + {"detail": "..."} 형태로 수렴한다 (500 금지).
- 교육 목적에 맞는 최소 구성: 인증은 JWT 가 아닌 세션 쿠키, 배포는 로컬 Docker 로 한정한다 (04 PRD §2-2 비목표 준수).

2. 전체 아키텍처

2-1. 논리 아키텍처 (계층도)





요청은 항상 브라우저 → FastAPI 라우터 → (인증 확인) → crud → SQLAlchemy → PostgreSQL 순으로 흐르고, 응답은 역순으로 반환된다. 라우터는 HTTP 프로토콜(상태 코드·요청/응답 스키마)만 담당하고, crud 는 DB 조작만 담당하여 서로의 관심사를 침범하지 않는다.

2-2. 배포 아키텍처 (로컬 개발 환경)

구성요소	실행 방식	포트/접속
PostgreSQL 16	Docker 컨테이너 (pg-lab)	5432 (호스트 매핑)
FastAPI 앱	uvicorn main:app --reload	8000 (http://127.0.0.1:8000)
정적 파일	FastAPI StaticFiles 마운트	/static/*
API 문서	FastAPI 자동 생성 (Swagger UI)	/docs

04 PRD §2-2(N4)에 따라 실서버 배포·HTTPS 는 범위 밖이며, 로컬 127.0.0.1 환경을 기준으로 설계한다. §9(향후 확장)에서 배포 확장 방향을 별도로 다룬다.

3. 기술 스택 및 버전

00 과제로 정의서에서 검증한 조합을 그대로 채택한다. 아래 버전은 실제로 함께 설치하여 충돌이 없음을 확인했다.

구성요소	버전	역할	선정 근거
Python	3.12.3	실행 환경	FastAPI 3.10+ 요구 충족, 표준 최신 안정 버전
FastAPI	0.139.0	웹 프레임워크(API 서버)	타입 힌트 기반 자동 검증·자동 문서화
Pydantic	2.13.4	입력 데이터 검증	FastAPI 표준 검증 엔진(v2)
SQLAlchemy	2.0.51	ORM	Mapped[...] 선언형 문법(2.0 스타일)
psycopg	3.3.4	PostgreSQL 드라이버	SQLAlchemy 2.0 과 호환되는 최신 3 세대 드라이버
pwdlib[argon2]	0.3.0	비밀번호 해싱(Argon2)	FastAPI 공식 문서 권장 해시 알고리즘

구성요소	버전	역할	선정 근거
Uvicorn	0.50.0	ASGI 서버	FastAPI 개발/실행 표준 서버
PostgreSQL	16 (Docker)	데이터베이스	관계형 DB 표준, FK/UNIQUE 제약 지원
python-dotenv	1.0.1	.env 환경변수 로딩	§7-2 DATABASE_URL 을 .env 로 분리하려면 필요 (없으면 KeyError)

3-1. 인증 방식 선정 근거

세션 쿠키 로그인을 채택한다. 근거는 00 문서와 동일하다: 추가 라이브러리 불필요(secrets 표준 라이브러리), 브라우저가 쿠키를 자동 보관/첨부하여 프론트엔드 구현이 단순, Django/Flask 등 전통 프레임워크의 표준 방식. JWT 는 심화 과정(§9 향후 확장)으로 미룬다.

4. 데이터베이스 설계

4-1. ER 다이어그램 (텍스트)

```

users (1) —>< sessions (N)    users.id = sessions.user_id
users (1) —>< categories (N)   users.id = categories.user_id
users (1) —>< contacts (N)     users.id = contacts.user_id
categories (1) —>< contacts (N) categories.id = contacts.category_id

```

모든 화살표는 외래키(FK)이며, 1 명의 사용자가 여러 세션·카테고리·연락처를 가질 수 있는 1:N 관계다.

contacts 는 users 와 categories 양쪽에서 FK 를 받는 이중 종속 테이블로, 01 문서 §1-1 의 관계도를 DDL 수준으로 구체화한 것이다.

스키마의 단일 진실 원천(Source of Truth): 아래 §4-2 DDL 과 §4-3 SQLAlchemy 모델은 같은 스키마를 두 가지 표기로 보여주는 것으로, 실제 테이블은 SQLAlchemy 모델 기준으로 앱 시작 시 Base.metadata.create_all(engine) 이 생성한다. §4-2 DDL 은 수동으로 실행하지 않으며, 설계 의도를 SQL 로 읽기 쉽게 보여주는 참고 자료다. (수동 DDL 실행과 create_all() 을 동시에 쓰면 스키마 정의가 miss-match 될 수 있으므로 반드시 하나만 사용한다.)

4-2. DDL (참고용 — 실제 생성은 §4-3 SQLAlchemy 모델 기준)

```

CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(20) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT now()
);

CREATE TABLE sessions (
    session_id CHAR(64) PRIMARY KEY,
    user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    created_at TIMESTAMP NOT NULL DEFAULT now()
);

CREATE TABLE categories (
    id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    name VARCHAR(10) NOT NULL,
    CONSTRAINT uq_categories_user_name UNIQUE (user_id, name)
);

CREATE TABLE contacts (

```

```

    id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    category_id INTEGER NOT NULL REFERENCES categories(id) ON DELETE RESTRICT,
    name VARCHAR(5) NOT NULL,
    phone CHAR(11) NOT NULL,
    addr VARCHAR(255) NOT NULL DEFAULT '',
    CONSTRAINT uq_contacts_user_phone UNIQUE (user_id, phone)
);

```

```

-- user_id 단독 조회는 위 UNIQUE(user_id, ...) 복합 인덱스의 선행 컬럼으로 이미 커버되므로
-- 별도의 ix_contacts_user_id / ix_categories_user_id 인덱스는 만들지 않는다 (중복 방지).
CREATE INDEX ix_contacts_name ON contacts(name); -- 이름 검색(GET /contacts?name=)용

```

설계 포인트

- categories → contacts 는 ON DELETE RESTRICT 로 지정한다. 사용 중인 카테고리를 실수로 지워도 DB 가 원천 차단하며, 이 제약 위반을 애플리케이션이 409 로 감싸서 응답한다(01 문서 §3-3).
- users → sessions/categories/contacts 는 ON DELETE CASCADE 로 지정한다. (회원 탈퇴 기능은 이번 범위 밖이나, 데이터 정합성을 위해 규칙만 마련)
- sessions.session_id 는 CHAR(64) — secrets.token_hex(32)의 결과 길이(32 바이트 → 16 진수 64 자)에 정확히 맞춘다.

4-3. SQLAlchemy 2.0 모델 매핑 예시

```

class Contact(Base):
    __tablename__ = "contacts"
    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id", ondelete="CASCADE"))
    category_id: Mapped[int] = mapped_column(ForeignKey("categories.id",
ondelete="RESTRICT"))
    name: Mapped[str] = mapped_column(String(5))
    phone: Mapped[str] = mapped_column(String(11))
    addr: Mapped[str] = mapped_column(String(255), default="")
    category: Mapped["Category"] = relationship("Category") # ← v1.2 추가:
category_name 응답에 필요
    __table_args__ = (UniqueConstraint("user_id", "phone",
name="uq_contacts_user_phone"),)

```

v1.2 수정: §6-1 응답 스펙의 category_name 필드를 만들려면 contact.category.name 처럼 접근해야 하는데, Contact 모델에 category relationship 이 빠져 있으면 연락처 추가/조회 API(FR-05~08) 전체가 AttributeError 로 죽는다. session.user 와 동일한 패턴의 누락이므로 함께 점검하다 발견했다.

get_current_user()가 session.user 로 사용자를 바로 꺼내려면, LoginSession 모델에 relationship 을 반드시 선언해야 한다(FK 컬럼만으로는 .user 속성이 자동으로 생기지 않는다). 아래는 그 선언 예시다.

```

class User(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(primary_key=True)
    username: Mapped[str] = mapped_column(String(20), unique=True)
    password_hash: Mapped[str] = mapped_column(String(255))

class LoginSession(Base):
    __tablename__ = "sessions"
    session_id: Mapped[str] = mapped_column(String(64), primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id", ondelete="CASCADE"))
    user: Mapped["User"] = relationship("User") # ← 이 줄이 있어야 session.user 사용
가능

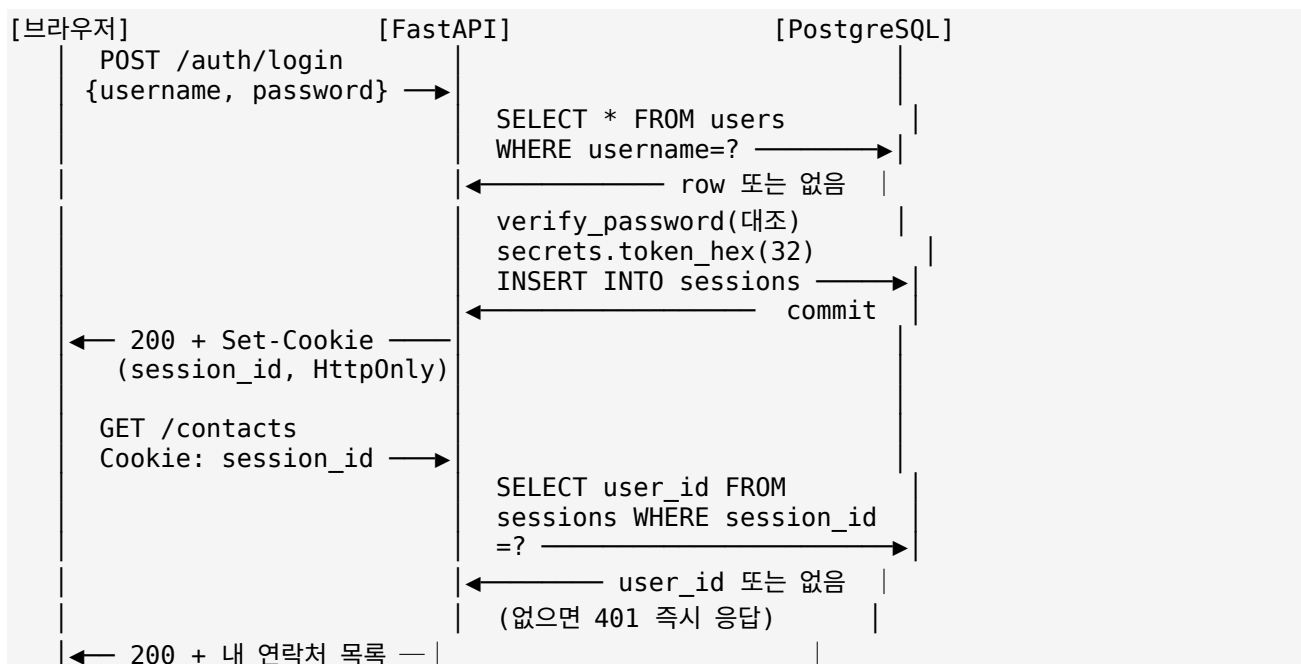
```

4-4. 트랜잭션 정책

시나리오	정책	구현
일반 CRUD 1 건 처리	요청 성공 시 즉시 commit	라우터/crud 단에서 commit 후 응답
회원가입 + 기본 카테고리 3 개 생성	하나의 트랜잭션으로 묶음	commit 전 실패 시 전체 rollback (반쪽 계정 방지)
처리 도중 예외 발생	rollback 후 적절한 상태 코드 응답	try/except + db.rollback()

5. 인증 설계 (세션 기반)

5-1. 인증 흐름 시퀀스



5-2. 쿠키 속성

속성	값	이유
HttpOnly	true	JavaScript(document.cookie)가 세션 값을 읽지 못하게 하여 XSS 로 인한 토큰 탈취 방지
Path	/	모든 API 경로에서 쿠키가 첨부되도록
SameSite	Lax	일반적인 동일 사이트 요청에서 자동 첨부, 외부 사이트발 위조 요청은 차단(CSRF 완화)
Secure	로컬 환경 false (배포 시 true 권장)	HTTPS 미사용 로컬 개발 환경 특성 반영 (§9 확장 시 true 로 전환)

5-3. 비밀번호 저장

비밀번호는 `pwplib(Argon2)`로 해싱하여 `password_hash` 컬럼에만 저장한다. 원문은 요청 처리 직후 메모리에서만 존재하고 어디에도 로그·저장되지 않는다.

```

from pwdlib import PasswordHash
password_hash = PasswordHash.recommended()

hashed = password_hash.hash(plain_password)      # 가입 시
ok = password_hash.verify(plain_password, hashed) # 로그인 시

```

5-4. get_current_user 의존성 (문지기)

모든 보호 API(FR-05~12)는 라우터 함수 인자에 다음을 선언하는 것만으로 인증이 적용된다. 이 함수가 쿠키 → 세션 조회 → 사용자 반환(또는 401)까지 전담한다.

```

def get_current_user(
    session_id: str | None = Cookie(default=None),
    db: Session = Depends(get_db),
) -> models.User:
    if session_id is None:
        raise HTTPException(401, "로그인이 필요합니다")
    session = db.get(models.LoginSession, session_id)
    if session is None:
        raise HTTPException(401, "로그인이 필요합니다")    # 세션 개념 자체에 만료가 없으므로 '
만료' 문구 사용 안 함
    return session.user    # LoginSession.user relationship (§4-3) 덕분에 바로 조회 가능

```

5-5. 데이터 격리 구현 규칙

모든 조회·수정·삭제 쿼리는 WHERE 절에 user_id 를 포함해야 한다. 이 규칙 하나로 "남의 데이터 접근 시 404"(01 문서 §5-2)가 자동으로 보장된다. 예외 없이 적용되어야 하므로, 코드 리뷰 체크리스트(부록 §10)에 별도 항목으로 명시한다.

6. API 설계 표준

6-1. 엔드포인트 전체 목록

메서드	경로	인증	설명	성공 코드
POST	/auth/signup	불필요	회원가입 + 기본 카테고리 3 개 생성	201
POST	/auth/login	불필요	로그인, 세션 발급 + 쿠키 설정	200
POST	/auth/logout	필요	세션 삭제 + 쿠키 삭제	200
GET	/auth/me	필요	현재 로그인 사용자 확인	200
GET	/contacts	필요	내 연락처 목록/검색/필터	200
POST	/contacts	필요	연락처 추가	201
PATCH	/contacts/{id}	필요	연락처 부분 수정	200
DELETE	/contacts/{id}	필요	연락처 삭제	204
GET	/categories	필요	카테고리 목록	200
POST	/categories	필요	카테고리 추가	201
PATCH	/categories/{id}	필요	카테고리 이름 수정	200
DELETE	/categories/{id}	필요	카테고리 삭제 (사용 중이면 409)	204

메서드	경로	인증	설명	성공 코드
GET	/	불필요	화면(index.html) 제공	200

6-2. 응답 스키마 규약

- 입력 스키마(...Create/...Update)와 출력 스키마(...Out)를 항상 분리한다 — id 위조·password_hash 노출 방지.
- 목록 응답은 {"total": N, "items": [...]} 형태로 통일한다.
- 오류 응답은 항상 {"detail": "..."} 단일 필드로 통일한다 (FastAPI HTTPException 표준).

6-3. 상태 코드 규약

코드	의미	발생 조건
200	성공(조회/수정/로그인 등)	정상 처리 완료, 응답 본문 있음
201	생성 성공	회원가입/연락처·카테고리 추가
204	성공, 본문 없음	삭제 성공
401	인증 실패	세션 쿠키 없음/무효, 로그인 정보 불일치
404	대상 없음	없는 id, 남의 데이터 id 접근
409	규칙 충돌	아이디/전화번호/카테고리명 중복, 사용 중 카테고리 삭제
422	형식 위반	Pydantic 자동 검증 실패
500	발생 금지	예외 처리 누락을 의미 — 반드시 제거

7. 모듈(코드) 설계

03 기능정의서의 파일 구조를 기술 관점에서 확정한다. 계층 간 의존 방향은 반드시 아래로만 흐른다: routers → crud → models/database. 역방향 의존(crud가 routers를 참조하는 등)은 금지한다.

```

contact_app/
├── main.py           # FastAPI 앱 생성, 라우터 등록, 정적 파일 마운트
├── database.py       # engine, SessionLocal, get_db()
├── models.py         # User, LoginSession, Category, Contact (ORM 모델)
├── schemas.py        # Pydantic 입출력 스키마
├── security.py       # hash_password, verify_password
├── crud.py           # DB 조작 함수 (라우터에서 호출)
├── routers/
│   ├── auth.py      # 인증 4개 엔드포인트 + get_current_user
│   ├── contacts.py  # 연락처 4개 엔드포인트
│   └── categories.py # 카테고리 4개 엔드포인트
├── static/
│   ├── index.html
│   └── app.js

```

7-1. 의존성 주입(Dependency Injection) 설계

의존성	제공하는 것	사용처
Depends(get_db)	요청 단위 DB 세션(Session)	모든 라우터 함수
Depends(get_current_user)	인증된 User 객체 (실패 시 401)	보호 API 8 개(FR-05~12)

7-2. 설정 관리

DB 접속 문자열 등 환경별로 달라지는 값은 코드에 하드코딩하지 않고 환경 변수(.env) 로 분리한다. .env 파일을 애플리케이션이 직접 읽으려면 python-dotenv(\$3 기술 스택에 추가)로 load_dotenv()를 호출해야 하며, 이를 생략하면 os.environ["DATABASE_URL"]에서 KeyError가 발생한다. (Docker Compose의 environment: 섹션으로 주입하는 경우는 불필요.)

```
# .env
DATABASE_URL=postgresql+psycopg://user:password@localhost:5432/contactdb

# database.py
from dotenv import load_dotenv
load_dotenv() # .env를 읽어 os.environ에 반영 (python-dotenv 필요)
engine = create_engine(os.environ["DATABASE_URL"])
```

8. 비기능 요구사항(NFR) 기술 대응

NFR	요구사항	기술적 구현 방안
NFR-01	어떤 요청에도 500 을 응답하지 않음	라우터마다 예외를 명시적으로 HTTPException으로 변환, 예상 못한 예외는 전역 exception handler로 500 대신 안내 메시지 반환
NFR-02	계층별 파일 분리	routers/crud/models/schemas 분리 (§7 모듈 설계)
NFR-03	비밀번호 원문 저장 금지	pwplib(Argon2) 해싱, password_hash 컬럼만 저장 (§5-3)
NFR-04	데이터 API 전부 로그인 필수 + 격리	get_current_user 의존성 + 모든 쿼리에 user_id 필터 (§5-4, §5-5)
NFR-05	한글 입출력 깨짐 없음	UTF-8 인코딩 전 구간 통일 (DB charset, FastAPI 응답 헤더, HTML meta charset)
NFR-06	/docs 자동 문서화	FastAPI 기본 제공(OpenAPI) 그대로 사용, 별도 구현 불필요

9. 향후 확장 시 기술 변경 사항 (참고)

04 PRD §11의 확장 항목을 기술적으로 구현할 때 바뀌는 지점을 미리 정리한다. 지금 구현할 범위는 아니며, 설계가 이 방향을 막지 않는지 확인하는 참고용이다.

확장 항목	기술적 변경 포인트
세션 만료 시간 도입	sessions 테이블에 expires_at 컬럼 추가, get_current_user에서 만료

확장 항목	기술적 변경 포인트
	시각 비교 로직 추가
JWT 방식 전환	sessions 테이블 대신 서명된 토큰 발급/검증 로직으로 대체, 쿠키 대신 Authorization 헤더 사용
실서버 배포	Secure 쿠키 속성 true 전환, HTTPS(리버스 프록시 또는 Uvicorn TLS), 환경변수 분리 강화
페이지네이션	GET /contacts 에 limit/offset(또는 cursor) 쿼리 파라미터 추가, count 쿼리 별도 실행
연락처 항목 확장(사진 등)	contacts 테이블에 컬럼 추가 + 마이그레이션 도구(Alembic) 도입 검토

10. 부록 — 구현 체크리스트 및 문서 추적

10-1. 보안·격리 코드 리뷰 체크리스트

- [] 모든 SELECT/UPDATE/DELETE 쿼리의 WHERE 절에 user_id 가 포함되어 있는가
- [] 비밀번호가 로그(print, 로깅 라이브러리)에 절대 출력되지 않는가
- [] 쿠키에 HttpOnly 속성이 설정되어 있는가
- [] 로그인 실패 응답이 "아이디 없음"과 "비밀번호 틀림"을 구분하지 않는가
- [] 남의 데이터 접근 시 403 이 아니라 404 로 응답하는가
- [] 카테고리 삭제 시 FK 위반(500)이 아니라 409 로 변환되는가

10-2. 문서 간 대응표

TRD 섹션	관련 문서	대응 항목
§4 DB 설계	01 구현요구사항 §1	테이블 정의(users/sessions/categories/contacts)
§5 인증 설계	00 §3-6, 01 §3-1, 03 FN-004~008	세션 로그인 흐름
§6 API 설계	01 §2~§3 (FR-01~13)	엔드포인트 명세
§7 모듈 설계	03 기능정의서 전체	파일/함수 구조
§8 NFR 대응	01 §7 (NFR-01~06)	비기능 요구사항
전체	04 PRD §9 (AC-01~17)	인수 기준 충족 여부 최종 검증

문서 읽는 순서(00 문서§10 과 동일): 과제목적 → 구현요구사항 → 화면정의서 → 기능정의서 → PRD → TRD(본 문서). TRD 는 앞선 5 개 문서의 "무엇을 왜" 를 "어떻게 구현하는가" 로 마무리하는 최종 기술 설계도 역할을 한다.